

MIPS 指令速查与详解

面向 MIPS / MARS 初学者的中文参考手册

使用范围

本手册以经典 MIPS32（常见于 MARS 的教学子集）为中心。前七章列出真实机器指令：它们有固定的 R、I 或 J 型 32 位编码。最后一章的伪指令由汇编器翻译为一条或多条真实指令，不能把它们当成单条 CPU 指令。

关于 syscall

MARS 中的 syscall 是模拟器提供的服务入口：通常先把服务号放入 \$v0、参数放入指定寄存器，再执行 syscall。它不是普通 MIPS32 CPU 指令，实际硬件的异常与操作系统接口更复杂。

阅读约定

\$d/\$s/\$t 表示目标/源/临时寄存器；imm 为 16 位立即数；offset(base) 为基址加偏移寻址。示例均假定标签和寄存器已按上下文初始化。

0. 编码、寄存器与立即数速查

R / I / J 型字段概览

类型	位域（从高到低）	典型指令	说明
R 型	opcode[31:26] rs[25:21] rt[20:16] rd[15:11] shamt[10:6] funct[5:0]	add, and, sll	opcode 通常为 0, funct 决定操作。
I 型	opcode[31:26] rs[25:21] rt[20:16] immediate[15:0]	addi, lw, beq	立即数按指令规则符号扩展或零扩展。分支偏移以指令字为单位。
J 型	opcode[31:26] target[25:0]	j, jal	目标实际为 {PC+4[31:28], target, 00}。

常用寄存器

\$0	别名	用途	MARS 中常见约定
\$0	\$zero	恒为 0	写入会被丢弃。
\$1	\$at	汇编器临时	伪指令可能占用；一般不要手用。
\$2-\$3	\$v0-\$v1	返回值	MARS syscall 也常用 \$v0 放服务号。
\$4-\$7	\$a0-\$a3	前四个参数	print_int 的参数在 \$a0。
\$8-\$15	\$t0-\$t7	临时寄存器	调用者保存。
\$16-\$23	\$s0-\$s7	保存寄存器	被调用者若使用应保存并恢复。
\$24-\$25	\$t8-\$t9	临时寄存器	调用者保存。
\$28	\$gp	全局指针	数据访问约定。
\$29	\$sp	栈指针	栈通常向低地址增长。
\$30	\$fp/\$s8	帧指针/保存	取决于调用约定。
\$31	\$ra	返回地址	jal 写入；jr \$ra 返回。
特殊	HI / LO	乘除结果寄存器	用 mfhi / mflo 读出。

立即数扩展与地址要点

类别	规则	涉及指令
符号扩展	16 位 imm 扩成 32 位并保留符号	addi/addiu, slti/sltiu, lb/lh/lw, sb/sh/sw, beq/bne
零扩展	高 16 位补 0	andi, ori, xori
装入高位	imm 直接成为结果的高 16 位，低 16 位为 0	lui
分支目标	PC + 4 + (signext(imm) << 2)	beq, bne, blez, bgtz
J 目标	{PC+4 的高 4 位, target, 00}	j, jal

1. 算术指令（真实机器指令）

add | Add | R: add \$d, \$s, \$t

作用：有符号加法；溢出时产生算术溢出异常。

```
add $t0, $t1, $t2 # t0 = t1 + t2
```

注意：教学中常用 add；若不希望因有符号溢出触发异常，可使用 addu/addiu。

addu | Add Unsigned | R: addu \$d, \$s, \$t

作用：32 位按模 2^{32} 相加，不报告溢出。

```
addu $t0, $t1, $t2
```

addi | Add Immediate | I: addi \$t, \$s, imm

作用：有符号立即数加法；imm 符号扩展，溢出会异常。

```
addi $t0, $t1, -4
```

注意：不是“无符号立即数”；立即数仍是符号扩展。

addiu | Add Immediate Unsigned | I: addiu \$t, \$s, imm

作用：按模 2^{32} 加法，不报告溢出；imm 仍符号扩展。

```
addiu $sp, $sp, -16
```

注意：常用于调整栈指针。

sub | Subtract | R: sub \$d, \$s, \$t

作用：有符号减法；溢出时异常。

```
sub $t0, $t1, $t2 # t0 = t1 - t2
```

subu | Subtract Unsigned | R: subu \$d, \$s, \$t

作用：按模 2^{32} 相减，不报告溢出。

```
subu $t0, $t1, $t2
```

2. 逻辑与移位（真实机器指令）

and | And | R: and \$d, \$s, \$t

作用：逐位与。

```
and $t0, $t1, $t2
```

andi | And Immediate | I: andi \$t, \$s, imm

作用：寄存器与零扩展的立即数逐位与。

```
andi $t0, $t1, 0x00FF
```

注意：andi 的 imm 零扩展，不能用它直接产生负掩码。

or | Or | R: or \$d, \$s, \$t

作用：逐位或。

```
or $t0, $t1, $t2
```

ori | Or Immediate | I: ori \$t, \$s, imm

作用：寄存器与零扩展立即数逐位或。

```
ori $t0, $zero, 0x00FF
```

xor | Exclusive Or | R: xor \$d, \$s, \$t

作用：逐位异或。

```
xor $t0, $t1, $t2
```

xori | Exclusive Or Immediate | I: xori \$t, \$s, imm

作用：寄存器与零扩展立即数异或。

```
xori $t0, $t1, 1
```

nor | Not Or | R: nor \$d, \$s, \$t

作用：逐位或后取反。

```
nor $t0, $t1, $t2
```

注意：nor \$d, \$s, \$zero 等价于 bitwise not \$s。

sll | Shift Left Logical | R: sll \$d, \$t, shamt

作用：逻辑左移固定位数，低位补 0。

```
sll $t0, $t1, 2 # 4■■■■■■■■
```

srl | Shift Right Logical | R: srl \$d, \$t, shamt

作用：逻辑右移固定位数，高位补 0。

```
srl $t0, $t1, 1
```

sra | Shift Right Arithmetic | R: sra \$d, \$t, shamt

作用：算术右移固定位数，高位复制符号位。

```
sra $t0, $t1, 1
```

注意：对有符号数的右移通常选 sra。

sllv | Shift Left Logical Variable | R: sllv \$d, \$t, \$s

作用：按 \$s 的低 5 位给出的位数逻辑左移。

```
sllv $t0, $t1, $t2
```

srlv | Shift Right Logical Variable | R: srlv \$d, \$t, \$s

作用：按 \$s 的低 5 位逻辑右移。

```
srlv $t0, $t1, $t2
```

srav | Shift Right Arithmetic Variable | R: srav \$d, \$t, \$s

作用：按 \$s 的低 5 位算术右移。

```
srav $t0, $t1, $t2
```

3. 比较与条件转移（真实机器指令）

slt | Set on Less Than | R: slt \$d, \$s, \$t

作用：若 $s < t$ （有符号）则 $d=1$ ，否则为 0。

```
slt $t0, $s0, $s1
```

sltu | Set on Less Than Unsigned | R: sltu \$d, \$s, \$t

作用：按无符号数比较，小于则置 1。

```
sltu $t0, $s0, $s1
```

注意：同一位模式在 `slt` 和 `sltu` 下可能得到不同结果。

slti | Set on Less Than Immediate | I: slti \$t, \$s, imm

作用：将 s 与符号扩展 imm 做有符号比较。

```
slti $t0, $s0, 10
```

sltiu | Set on Less Than Immediate Unsigned | I: sltiu \$t, \$s, imm

作用：将 s 与符号扩展 imm 做无符号比较。

```
sltiu $t0, $s0, 10
```

注意：名称带 `u` 只影响比较； imm 仍符号扩展。

beq | Branch if Equal | I: beq \$s, \$t, label

作用：若两寄存器相等则 PC 相对跳转。

```
beq $t0, $zero, done
```

bne | Branch if Not Equal | I: bne \$s, \$t, label

作用：若两寄存器不等则跳转。

```
bne $t0, $t1, loop
```

blez | Branch if Less Than or Equal to Zero | I: blez \$s, label

作用：若 s （有符号）小于等于 0 则跳转。

```
blez $t0, nonpositive
```

bgtz | Branch if Greater Than Zero | I: bgtz \$s, label

作用：若 s （有符号）大于 0 则跳转。

```
bgtz $t0, positive
```

提示：MARS 默认通常不模拟早期 MIPS 的“延迟槽”执行语义；写课程代码时仍应以课程/教师给定设置为准。

4. 立即数、地址构造与访存（真实机器指令）

lui | Load Upper Immediate | I: lui \$t, imm

作用：把 imm 放入目标寄存器高 16 位，低 16 位清零。

```
lui $t0, 0x1234 # t0 = 0x12340000
```

lb | Load Byte | I: lb \$t, offset(\$s)

作用：从内存取 8 位并符号扩展到 32 位。

```
lb $t0, 0($s0)
```

注意：若需要 0..255 而非负数，使用 lbu。

lbu | Load Byte Unsigned | I: lbu \$t, offset(\$s)

作用：从内存取 8 位并零扩展。

```
lbu $t0, 0($s0)
```

lh | Load Halfword | I: lh \$t, offset(\$s)

作用：取 16 位并符号扩展。

```
lh $t0, 2($s0)
```

注意：地址应按 2 字节对齐。

lhu | Load Halfword Unsigned | I: lhu \$t, offset(\$s)

作用：取 16 位并零扩展。

```
lhu $t0, 2($s0)
```

lw | Load Word | I: lw \$t, offset(\$s)

作用：从内存取 32 位字。

```
lw $t0, 4($sp)
```

注意：地址应按 4 字节对齐。

sb | Store Byte | I: sb \$t, offset(\$s)

作用：将 \$t 低 8 位写入内存。

```
sb $t0, 0($s0)
```

sh | Store Halfword | I: sh \$t, offset(\$s)

作用：将 \$t 低 16 位写入内存。

```
sh $t0, 2($s0)
```

sw | Store Word | I: sw \$t, offset(\$s)

作用：将 32 位字写入内存。

```
sw $ra, 12($sp)
```

注意：标准函数序言/尾声经常保存与恢复 \$ra、\$s 寄存器。

5. 跳转、调用、乘除与 HI/LO（真实机器指令）

j | Jump | J: j label

作用：无条件跳到标签地址。

```
j loop
```

jal | Jump and Link | J: jal label

作用：跳转并把返回地址写入 \$ra。

```
jal print_sum
```

注意：被调用函数用 jr \$ra 返回；嵌套调用前通常要保存 \$ra。

jr | Jump Register | R: jr \$s

作用：跳到寄存器内的地址。

```
jr $ra
```

jalr | Jump and Link Register | R: jalr \$d, \$s

作用：跳到寄存器地址，并把返回地址写到 \$d（默认常用 \$ra）。

```
jalr $ra, $t9
```

mult | Multiply | R: mult \$s, \$t

作用：有符号 32×32 位乘，64 位积写 HI:LO。

```
mult $t0, $t1  
mflo $t2 # ■■ 32 ■
```

multu | Multiply Unsigned | R: multu \$s, \$t

作用：无符号乘，结果写 HI:LO。

```
multu $t0, $t1  
mfhi $t2 # ■■ 32 ■
```

div | Divide | R: div \$s, \$t

作用：有符号除；商进 LO，余数进 HI。

```
div $t0, $t1  
mflo $t2 # ■  
mfhi $t3 # ■■
```

注意：除数不能为 0；不要假定所有边界情形在真实硬件上的行为。

divu | Divide Unsigned | R: divu \$s, \$t

作用：无符号除；商进 LO，余数进 HI。

```
divu $t0, $t1  
mflo $t2
```

mfhi | Move From HI | R: mfhi \$d

作用：读取 HI 到通用寄存器。

```
mfhi $t0
```

mflo | Move From LO | R: mflo \$d

作用：读取 LO 到通用寄存器。

```
mflo $t0
```


mthi | Move To HI | R: mthi \$s

作用：把通用寄存器写入 HI。

```
mthi $t0
```

mtlo | Move To LO | R: mtlo \$s

作用：把通用寄存器写入 LO。

```
mtlo $t0
```

6. 异常相关与 MARS syscall 服务

syscall | System Call (exception instruction) | R/特殊编码：syscall

作用：触发系统调用异常机制。MARS 捕获它并按 \$v0 中的服务号提供模拟器服务。

```
li $v0, 1 # MARS: ■■■■  
li $a0, 42  
syscall
```

注意：这是 MARS 模拟器服务入口，不是“打印”这类普通 CPU 指令。服务号和可用功能以 MARS 帮助为准。

break | Breakpoint | R/特殊编码：break

作用：触发断点异常，便于调试器/异常处理。

```
break
```

eret | Exception Return | 特权指令：eret

作用：从异常返回（协处理器 0 相关）；普通 MARS 用户程序通常不使用。

```
eret
```

注意：受异常处理与特权模式约束。MARS 教学代码一般无需编写。

MARS 常见 syscall 速查

\$v0	服务 (MARS)	参数 / 返回值	最小示例
1	打印整数	\$a0 = 整数	li \$v0,1; li \$a0,42; syscall
4	打印字符串	\$a0 = 字符串地址	li \$v0,4; la \$a0,msg; syscall
5	读整数	返回 \$v0	li \$v0,5; syscall; move \$t0,\$v0
8	读字符串	\$a0=缓冲区, \$a1=长度	li \$v0,8; la \$a0,buf; li \$a1,32; syscall
9	sbrk 分配堆空间	\$a0 = 字节数；地址返回 \$v0	li \$v0,9; li \$a0,16; syscall
10	退出	无	li \$v0,10; syscall
11	打印字符	\$a0 = 字符码	li \$v0,11; li \$a0,65; syscall
12	读字符	字符码返回 \$v0	li \$v0,12; syscall
13/14/15/16	文件打开/读/写/关闭	详见 MARS Syscall Help	仅 MARS 服务，非 CPU ISA
17	带状态退出	\$a0 = 状态	li \$v0,17; li \$a0,0; syscall

7. 常见伪指令（由汇编器展开，不是硬件单指令）

MARS 接受的伪指令会被汇编器改写。展开方式取决于操作数、地址和设置，下面的“可能展开”只用于理解；不要依赖某一种固定展开。

move | Move | 伪：move \$d, \$s

作用：复制寄存器值。通常展开为 `addu $d,$s,$zero` 或 `or`。

```
move $t0, $t1
```

li | Load Immediate | 伪：li \$t, constant

作用：装入常数；小常数可能为 `addiu/ori`，大常数常需 `lui` 加 `ori`。

```
li $t0, 0x12345678
```

注意：不要假定 `li` 永远只占一条指令。

la | Load Address | 伪：la \$t, label

作用：装入标签地址。

```
la $a0, message
```

注意：地址可能需要两条或更多真实指令；不要与 `lw` 混淆：`la` 取地址，`lw` 取地址处的内容。

nop | No Operation | 伪：nop

作用：不做有效操作；通常编码为 `sll $zero,$zero,0`。

```
nop
```

not | Not | 伪：not \$d, \$s

作用：逐位取反；通常为 `nor $d,$s,$zero`。

```
not $t0, $t1
```

neg | Negate | 伪：neg \$d, \$s

作用：有符号取负；通常为 `sub $d,$zero,$s`。

```
neg $t0, $t1
```

注意：若关心溢出，展开指令的 `add/sub` 语义仍重要。

negu | Negate Unsigned | 伪：negu \$d, \$s

作用：按模 2^{32} 取负；通常为 `subu`。

```
negu $t0, $t1
```

mul | Multiply (low word) | 伪：mul \$d, \$s, \$t

作用：计算乘积低 32 位；传统展开为 `mult` 后 `mflo`。

```
mul $t0, $t1, $t2
```

注意：不要与真实的 `mult/multu` 混为一谈；后者写 `HI/LO`。

rem | Remainder | 伪：rem \$d, \$s, \$t

作用：计算有符号余数；传统展开为 `div` 后 `mfhi`。

```
rem $t0, $t1, $t2
```

b | Branch | 伪：b label

作用：无条件分支；常展开为 `beq $zero,$zero,label`。

```
b loop
```

beqz | Branch if Equal Zero | 伪：beqz \$s, label

作用：若寄存器为 0 则跳转。

```
beqz $t0, done
```

bnez | Branch if Not Equal Zero | 伪：bnez \$s, label

作用：若寄存器不为 0 则跳转。

```
bnez $t0, loop
```

blt / bgt | Branch Less/Greater Than | 伪：blt \$s,\$t,label

作用：有符号比较后条件跳转，常借助 \$at 与 slt。

```
blt $t0, $t1, smaller
```

注意：可能占两条以上且使用 \$at；与 sltu 语义不同。

ble / bge | Branch Less/Greater Equal | 伪：bge \$s,\$t,label

作用：有符号比较后条件跳转。

```
bge $t0, $t1, ge_case
```

abs | Absolute Value | 伪：abs \$d, \$s

作用：计算绝对值，通常需分支与取负。

```
abs $t0, $t1
```

注意：最小负数的绝对值不能以有符号 32 位正数表示。

seq / sne | Set Equal / Not Equal | 伪：seq \$d,\$s,\$t

作用：比较相等/不等，结果置 0 或 1。

```
seq $t0, $t1, $t2
```

sle / sge | Set Less Equal / Greater Equal | 伪：sle \$d,\$s,\$t

作用：有符号比较，结果置 0 或 1。

```
sge $t0, $t1, $t2
```

8. 迷你模板与易错点

一个典型的 MARS 程序骨架

```
.data
message: .asciiz "Hello, MIPS!\n"
.text
.globl main
main:
li $v0, 4
la $a0, message
syscall
li $v0, 10
syscall
```

最后检查清单

问题	应记住的结论
add 与 addu	前者有符号溢出会异常；后者只是 32 位模运算。
lb 与 lbu	lb 符号扩展，lbu 零扩展。lh/lhu 同理。
sra 与 srl	sra 保留符号，srl 高位补零。
slt 与 sltu	一个按有符号比较，一个按无符号比较。
la 与 lw	la 得到地址；lw 读取该地址的一个 32 位字。
j 与 jal	jal 额外写 \$ra，支持函数返回。
mult/div 与 mul/rem	mult/div 是真实指令并写 HI/LO；mul/rem 在传统 MIPS 汇编中常是伪指令。
syscall	MARS 模拟器服务入口；先设置 \$v0 和参数寄存器。
伪指令	可能展开为多条真实指令并占用 \$at；查看 MARS 的 Basic/Expanded 指令显示可验证。

版本说明：面向 MARS 常见教学用法的基础速查。MIPS 有多种版本、厂商扩展与特权体系；课程若指定特定 ISA 或 MARS 设置，应以其文档为准。